Department of Computer Science
Technical University of Cluj-Napoca

# Energy Management System

Name: Balas Andrei-Ioan

# Contents

**1 Introduction** **3**

**2 Objectives** **3**

**3 Architecture** **3**

3.1 Frontend . . . 3

3.1.1 User Interface Design . . . 3

3.1.2 Authentication . . . 3

3.1.3 Client Features . . . 4

3.2 Microservices . . . 4

3.2.1 User Management Service . . . 4

3.2.2 Device Management Service . . . 4

3.2.3 Monitoring and Communication Management Service . . . 5

3.2.4 Chat Microservice . . . 5

3.2.5 Authorization Component: . . . 5

3.3 Conceptual architecture diagram . . . 6

3.4 Deployment diagram . . . 7

# 1 Introduction

The Monitoring and Communication Microservice represents a critical extension to the Energy Management System (EMS), introducing capabilities for asynchronous data processing and real-time notifications. Its primary role is to monitor energy usage in real time, process data from smart metering devices, and alert users when predefined thresholds are exceeded.

The system is built using a distributed architecture that emphasizes scalability, fault tolerance, and asynchronous communication. It integrates with the existing EMS infrastructure, leveraging RabbitMQ for message-oriented communication and WebSockets for real-time notifications.

The features implemented in this assignment provide users with enhanced energy insights, promoting efficient energy usage and timely responses to abnormal consumption patterns.

# 2 Objectives

- Implement a smart metering device simulator to produce energy consumption data.
- Use RabbitMQ as a message broker for asynchronous communication.
- Process hourly energy consumption data in the Monitoring and Communication Microservice.
- Notify users asynchronously when consumption exceeds thresholds.
- Integrate the new features into the EMS and provide historical energy data visualization.

# 3 Architecture

The Energy Management System (EMS) is designed using a microservice architecture, which promotes the development of independent, loosely coupled services that can communicate over a network. This architectural style enhances scalability, resilience, and maintainability, allowing each service to evolve independently and be deployed or updated without affecting the entire system. The EMS architecture consists of a frontend user interface and three core microservices: the User Management Service and the Device Management Service and the Monitoring-Communication Management Service.

## 3.1 Frontend

The frontend of the Energy Management System (EMS) is a critical user-facing component, developed using React.js, a powerful JavaScript library for building dynamic user interfaces. The design prioritizes responsiveness, intuitiveness, and user engagement, ensuring that both administrators and clients can efficiently navigate the system.

### 3.1.1 User Interface Design

The user interface is meticulously crafted with user experience at its core. Key features include:

- **Responsive Layout:** The frontend adapts seamlessly to various screen sizes, providing an optimal viewing experience on desktops, tablets, and mobile devices. This ensures accessibility for all users, regardless of their device.
- **Clear Navigation:** An intuitive navigation menu allows users to easily access different sections of the EMS, such as user management for administrators and device monitoring for clients. Dropdown menus and breadcrumbs enhance usability by guiding users through the application.

### 3.1.2 Authentication

- **Secure Login Process:** Users authenticate through a secure login form, which verifies credentials against the User Management Service. This process utilizes industry-standard practices such as hashing passwords and implementing JSON Web Tokens (JWT) for session management.
- **Role-Based Access Control:** Upon successful login, users are directed to different dashboards based on their roles. Administrators gain access to advanced functionalities for managing user accounts and devices, while clients are directed to their personal dashboards to monitor their energy consumption.

### 3.1.3 Client Features

- **Profile Management:** Clients can easily update their personal information, including contact details and preferences. This feature allows for a personalized experience within the EMS.
- **Data Visualization:** Clients can view all their devices that mapped to their account and informations about them. Clients can view line charts regarding the consumption of the devices in specified day. Clients get notified when the consumption of a device is above tha maximum hourly consumption of that device.
- **Chat:** Customer support chat, every user can enter to a customer support chat to talk to the admin.

## 3.2 Microservices

The Energy Management System (EMS) utilizes two dedicated microservices, each designed to handle specific functionalities that significantly enhance the system's overall performance, scalability, and efficiency. These microservices are the User Management Service and the Device Management Service.

### 3.2.1 User Management Service

The Energy Management System (EMS) utilizes two dedicated microservices, each designed to handle specific functionalities that significantly enhance the system's overall performance, scalability, and efficiency. These microservices are the User Management Service and the Device Management Service.

- **Account Management:** his service allows administrators to create new user accounts, retrieve user details, update existing account information, and delete accounts as necessary. The ability to perform these operations ensures that user data remains current and relevant.
- **Data Management:** The User Management Service maintains a secure database of user information, which includes unique user IDs, names, roles (administrator or client), and their associations with devices. Utilizing secure authentication methods, such as JSON Web Tokens (JWT), the service ensures that sensitive user data is protected against unauthorized access.
- **Role Management:** Administrators can assign and modify user roles within the system. This role-based access control dictates the permissions available to each user, ensuring that clients have appropriate access to their data while restricting sensitive administrative functionalities to authorized personnel.
- **Synchronization with Device Management:** When a user is deleted from the User Management Service, a message is sent to the Device Management Service via RabbitMQ, a message broker that facilitates communication between microservices. This message triggers the Device Management Service to delete all device mappings and related data associated with the deleted user. This ensures data consistency across the system and prevents orphaned records.

### 3.2.2 Device Management Service

The Device Management Service is tasked with overseeing the entire lifecycle of smart energy metering devices, providing administrators with the tools necessary to perform CRUD operations related to these devices.

- **Device Registration:** Administrators can easily add new smart devices to the system, ensuring clients have access to the latest energy monitoring technology. Each device is registered with a unique ID, along with detailed specifications such as description, address, and maximum hourly energy consumption, facilitating effective tracking and management.
- **User-Device Association:** The Device Management Service manages the relationships between users and their devices, enabling clients to view and control the devices they own. This association is crucial for providing clients with tailored insights into their energy usage, allowing them to monitor performance and make informed decisions about energy consumption.
- **Message Handling:** Upon receiving a deletion message from the User Management Service via RabbitMQ, the Device Management Service processes this notification and performs the necessary operations to remove all mappings and data related to the deleted user. This automated synchronization mechanism enhances the integrity of the system by ensuring that no outdated or irrelevant data persists.

### 3.2.3 Monitoring and Communication Management Service

The Monitoring and Communication Microservice adds real-time monitoring and notification features to the EMS, processing energy data and ensuring users are informed about critical consumption levels.

- **Energy Data Processing:** Receives energy measurements from the Smart Metering Device Simulator via RabbitMQ, computes hourly energy totals, and stores them in the database.
- **Real-Time Notifications:** One of the core functionalities of the microservice is its ability to notify users in real time when their energy consumption exceeds predefined limits. Notifications are sent via WebSockets, ensuring instant delivery to the user's web interface. This immediate feedback mechanism is critical for enabling users to respond quickly to abnormal energy consumption, thereby preventing potential issues or excessive costs.
- **Integration and Synchronization:** The microservice ensures synchronization with the Device Management Service through RabbitMQ. Changes to device data, such as the addition or deletion of devices, are communicated via message queues to keep the monitoring database up to date. This synchronization maintains data consistency across the system and supports efficient monitoring.
- **Visualization and Reporting:** The system supports the visualization of energy consumption trends on the client interface. Users can select specific days to view their hourly energy usage in the form of line or bar charts, offering a clear and intuitive representation of their consumption patterns. This feature provides valuable insights, helping users manage their energy usage more effectively.

### 3.2.4 Chat Microservice

This microservice facilitates communication between users and system administrators. Users can ask questions and receive responses in real-time, fostering better support and engagement. Messages are stored in a secure database for future reference, ensuring accountability and a complete history of interactions.

### 3.2.5 Authorization Component:

The authorization system ensures secure access to all EMS microservices by implementing role-based access control (RBAC). It uses robust authentication methods, such as JSON Web Tokens (JWT), to validate users and enforce permissions based on their roles, protecting sensitive data and ensuring only authorized users can perform specific actions.

## 3.3 Conceptual architecture diagram

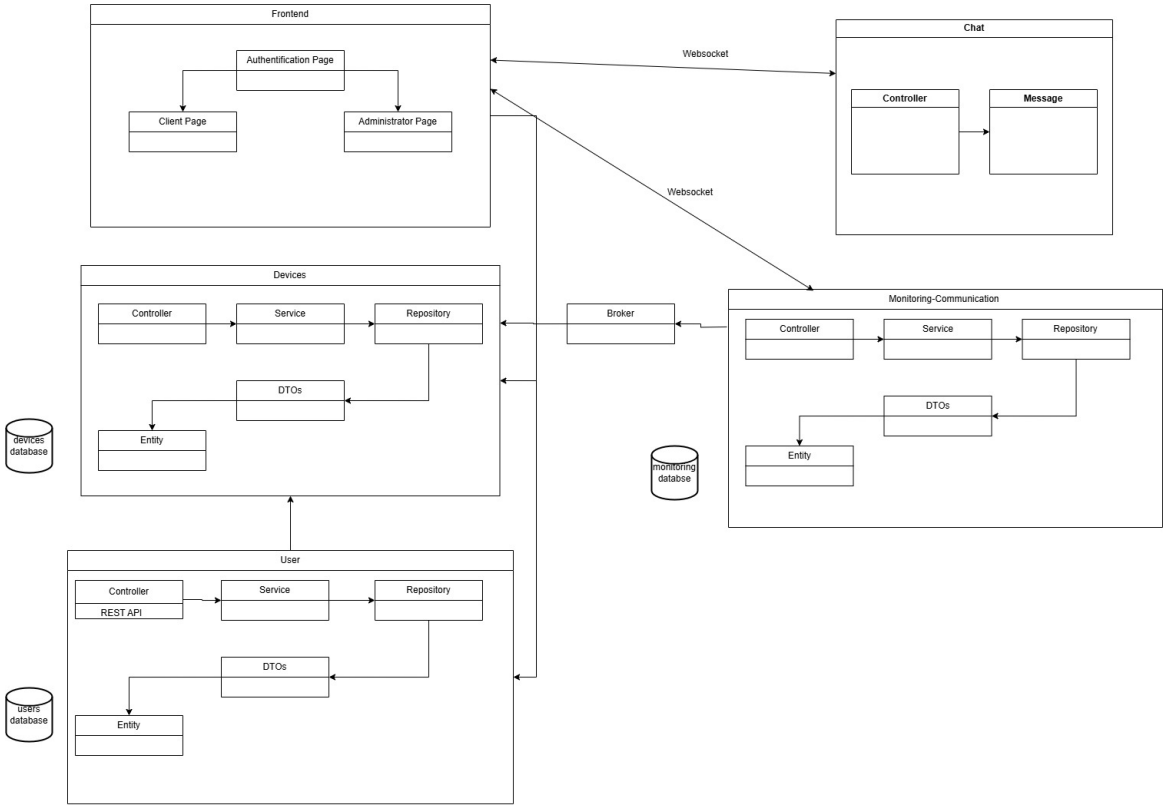


Figure 1: Conceptual architecture diagram

## 3.4 Deployment diagram

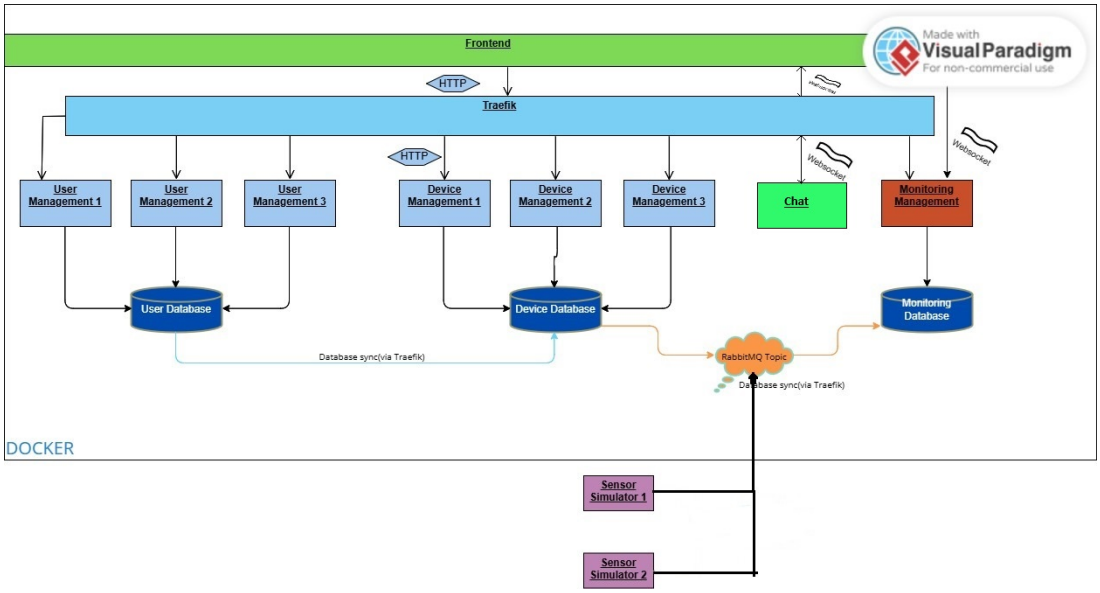


Figure 2: Deployment diagram